

Prolog

PROgramming in **LOGic**



BY FELIPE CARVALHO, GUSTAVO DIAS, THIAGO GOMES

Surgimento do Prolog



CRIADOR

Alain Colmerauer

CO-CRIADOR/CONTRIBUINTE

Philippe Roussel/Robert Kowalski

LOCAL DE SURGIMENTO

Universidade de Aix-Marseille

ANO DE CRIAÇÃO

1972

Qual era o objetivo do Prolog?

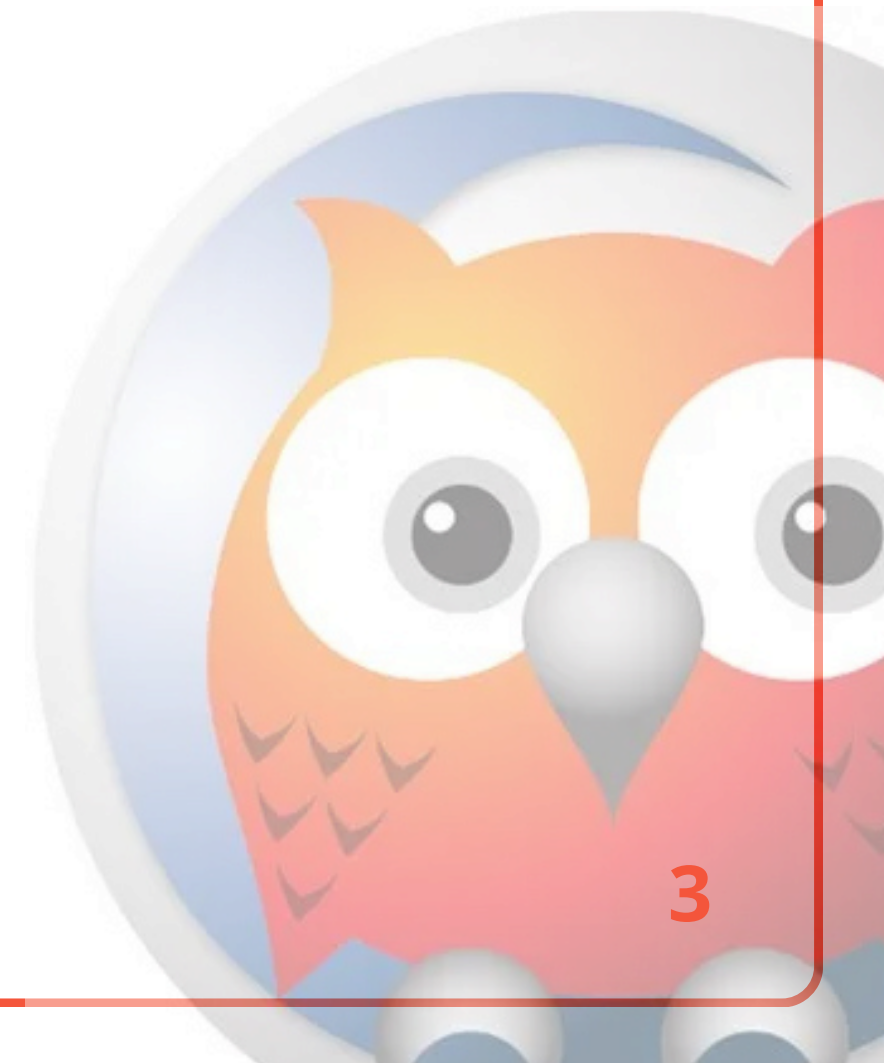
MOTIVAÇÃO PARA SUA CRIAÇÃO

Dificuldade de comunicação de seu criador com o idioma francês

OBJETIVOS FINAIS

Conversar com o computador

Criar um universo em que o computador pudesse responder



Linha do tempo



1972 - Criação do Prolog



1973–1977 - Criação do Prolog II e Expansão na Europa



1981 - Explosão do Prolog (Governo japonês cria o Projeto Quinta Geração (FGCS))



1995 - Início da padronização do Prolog



Dias atuais - Uso acadêmico em IA's



Paradigma LÓGICO

Prolog segue o paradigma da programação lógica, que é um subtipo do paradigma declarativo.

- Baseado em fatos e regras
- Execução como prova
- Inferência automática
- Backtracking embutido
- Unificação
- Ordem não imperativa

O que é o Paradigma Lógico?

O paradigma lógico é baseado na lógica formal (lógica de predicados de 1ª ordem). Em vez de dizer como resolver um problema (como fazemos na programação imperativa), você declara o que é verdade sobre o domínio do problema e o interpretador lógico (no caso, o Prolog) descobre como satisfazer as consultas com base nisso.

Semântica

SEMÂNTICA OPERACIONAL (RESOLUÇÃO)

COMO PROLOG EXECUTA UM PROGRAMA PARA
RESPONDER A CONSULTAS
MECANISMO PRINCIPAL:

RESOLUÇÃO POR UNIFICAÇÃO + BUSCA EM
PROFUNDIDADE COM BACKTRACKING.

ALGORITMO: SLD-RESOLUTION (SELECTIVE
LINEAR DEFINITE CLAUSE RESOLUTION).

ETAPAS DE EXECUÇÃO:

1. CONSULTA: O USUÁRIO FAZ UMA PERGUNTA.
2. UNIFICAÇÃO: PROLOG PROCURA CLÁUSULAS QUE PODEM SER UNIFICADAS COM O OBJETIVO ATUAL.
3. RESOLUÇÃO: SUBSTITUI O OBJETIVO PELO CORPO DA REGRA CORRESPONDENTE.
4. BUSCA EM PROFUNDIDADE: VAI EXPLORANDO A ÁRVORE DE PROVAS ATÉ ENCONTRAR UMA SOLUÇÃO (OU FALHAR).
5. BACKTRACKING: SE UMA TENTATIVA FALHA, VOLTA AO PONTO DE ESCOLHA ANTERIOR E TENTA OUTRA ALTERNATIVA.

SEMÂNTICA AXIOMÁTICA (LÓGICA DE PREDICADOS)

EM PROLOG, UM
PROGRAMA É UMA
BASE DE
CONHECIMENTO
COMPOSTA POR FATOS
E REGRAS.
ESSAS CLÁUSULAS
SÃO SENTENÇAS DA
LÓGICA DE
PREDICADOS.

Exemplo:

```
6 mulher(pam) .  
7 mulher(ann) .  
8 mulher(liz) .  
9 mulher(pat) .  
10 homem(tom) .  
11 homem(bob) .  
12 homem(jim) .  
13 genitor(pam, bob) .  
14 genitor(tom, bob) .  
15 genitor(tom, liz) .  
16 genitor(bob, ann) .
```

```
18 ?- mulher(pam) .  
  
19 ?- mulher(X) .  
X = pam ;  
X = ann ;  
X = liz ;  
X = pat ;  
false.
```

Sintaxe

TIPOS DE DADOS

Variáveis

Pode representar qualquer valor, seja um número, um átomo, uma lista, são iniciadas com maiúsculo ou (_). Ex: X, Nome, ALTURA, _Peso.

Inteiros

Qualquer sequencia numérica que não possui um (.). Ex: 0, 1, 13, 65.

Átomos

símbolos que representam constantes ou nomes, devem começar com letra minúscula ou estar entre aspas simples. Ex: joao, 'Joao', '19'.

Floats

Números com um ponto (.) e pelo menos uma casa decimal. Ex: 1.3, 23.89.

Listas

Sequencia ordenada de elementos entre [] e separados por virgulas. Ex: [1, 2, 3], [2, [2,3] , 7].

```
Lista = [1,2],  
nth1(1, Lista, Elem),  
write(Elem), nl.
```

Saída: 1

Sintaxe

- Operadores Lógicos:

(:-) -> equivalente à implicação(Se);

(;) -> equivalente a conjunção (Ou);

(,) -> equivalente a disjunção (E);

- Comentários:

```
% comentarios (%).  
/* comentarios.*/
```

- Fatos:

Representam informações básicas ou verdades atemporais. Sempre verdadeiros no contexto do programa.

Ex: fato(arg1, arg2, ...).

- Regras:

Representam conhecimento derivado, ou seja, verdade condicional.

Ex: consequente :- antecedente1, antecedente2,

Sintaxe

Entrada e Saída:

- write() → Comando de saída.
- read() → Comando de entrada.

Ex:

```
main:-
```

```
write('Digite um numero: '),nl,  
read(X), nl,  
fatorial(X, R),  
format('O Fatorial de ~w e igual a ~w~n', [X, R]).  
%halt.
```

Amarração e atribuição:

- (=) → Ele atribui o que foi colocado depois dele, sem avaliar nada;
- (is) → Primeiro ele faz uma avaliação aritmética, e depois atribui.

Ex:

```
main :-  
    X = 2+3,  
    write(X), nl,  
    Y = 4+9,  
    write(Y), nl.
```

Sintaxe

Modularização:

Um modulo é definido com o nome do modulo + `:-`, seguido das instruções do módulo.

Ex:

```
main :-  
    read(X),  
    read(Y),  
    soma(X, Y, R),  
    write(R), nl.  
  
soma(A, B, R) :-  
    R is A + B.  
  
:-main.
```

Repeat e Fail:

- fail → Sempre falha quando chamado. Força o backtracking, voltar atrás e tentar outra alternativa.
- repeat → Sempre tem sucesso quando chamado. E no backtracking, ele sempre volta a ter sucesso

Ex:

```
mostrar_alunos:-  
    aluno(Aluno, _),  
    write(Aluno), nl,  
    fail.
```

Sintaxe

ARITMÉTICA

Prolog é mais indicada para resolução de problemas simbólicos, mas também oferece suporte à aritmética.

Podemos utilizar duas notações para representar expressões em Prolog:

Infixa: $2 * a + b * c$ Prefixa: $+(*(2, a), (b, c))$

Operador	Significado
+, -, *, /	Realizar soma, subtração, multiplicação e divisão, respectivamente
is_	atribui uma expressão numérica à uma variável
mod	Obter o resto da divisão
^	Calcular potenciação
cos, sin, tan	Função cosseno, seno e tangente, respectivamente
exp	exponenciação
ln, log	logaritmo natural e logaritmo
sqrt	raiz

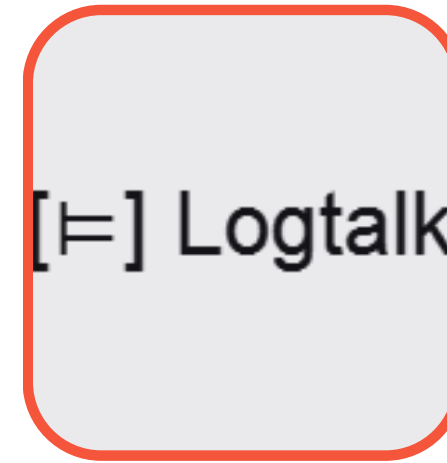
Linguagens Relacionadas



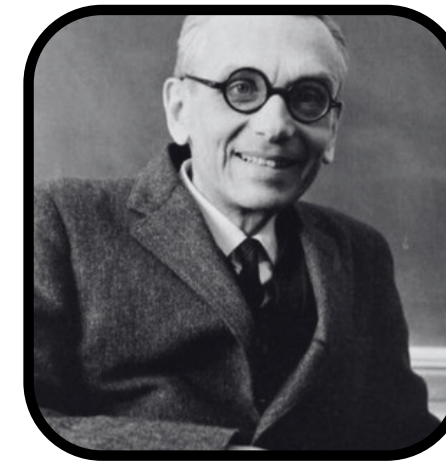
MERCURY



CIAO



LOGTALK



GÖDEL



TURBO
PROLOG



VISUAL
PROLOG



DATALOG

Uso da Linguagem

- **INTELIGÊNCIA ARTIFICIAL:**

Prolog é uma das linguagens clássicas para Inteligência Artificial, principalmente para representação de conhecimento, raciocínio e inferência lógica.

- **PROCESSAMENTO DE LINGUAGEM NATURAL (PLN):**

Prolog é muito usado em processamento de linguagem natural devido à sua capacidade de representar regras gramaticais e inferir relações.

- **JOGOS:**

Prolog é muito útil para modelar regras e estratégias de jogos. Como no xadrez, damas, sudoku, tomadas de decisão de NPCs.

Japanese Fifth Generation Computer Systems⁴⁴

- O projeto Fifth Generation Computer Systems foi uma iniciativa lançada pelo governo do Japão no início da década de 1980, com o objetivo de posicionar o país na vanguarda da computação e da inteligência artificial;
- Desenvolver a chamada "quinta geração de computadores";
- O projeto ficou ativo de 1982 a 1992, coordenado pelo Institute for New Generation Computer Technology (ICOT), em Tóquio;
- A FGCS usou o Prolog como linguagem base, por ser declarativa e permitir a representação de conhecimento por meio de fatos e regras lógicas;
- Apesar dos investimentos bilionários e de alguns avanços importantes em linguagens de programação lógica, em computação paralela e no desenvolvimento de ferramentas de IA simbólica, o FGCS não atingiu seus objetivos originais

Ambientes para Prolog: SWI-Prolog & VS Code



SWI Prolog

SWI PROLOG:

SWI-Prolog é um interpretador e ambiente de desenvolvimento para Prolog, mantido pela Universidade de Amsterdam.



VS CODE:

O VS Code pode também ser usado como ambiente para desenvolvimento em Prolog.

EXTENSÕES:



PROLOG



VSC-PROLOG

Exemplo:

```
fatorial.pl
File Edit Browse Compile Prolog Pce Help
fatorial.pl
main:-
    write('Digite um numero: '),nl,
    read(X), nl,
    fatorial(X, R),
    format('O Fatorial de ~w e igual a ~w~n', [X, R]).
    %halt.

fatorial(0, 1). %fato (usado para o caso base)

fatorial(N, R):-
    N > 0,
    N1 is N - 1,
    fatorial(N1, R1),
    R is N * R1.

%:-main. %chamada de diretiva.
```

```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

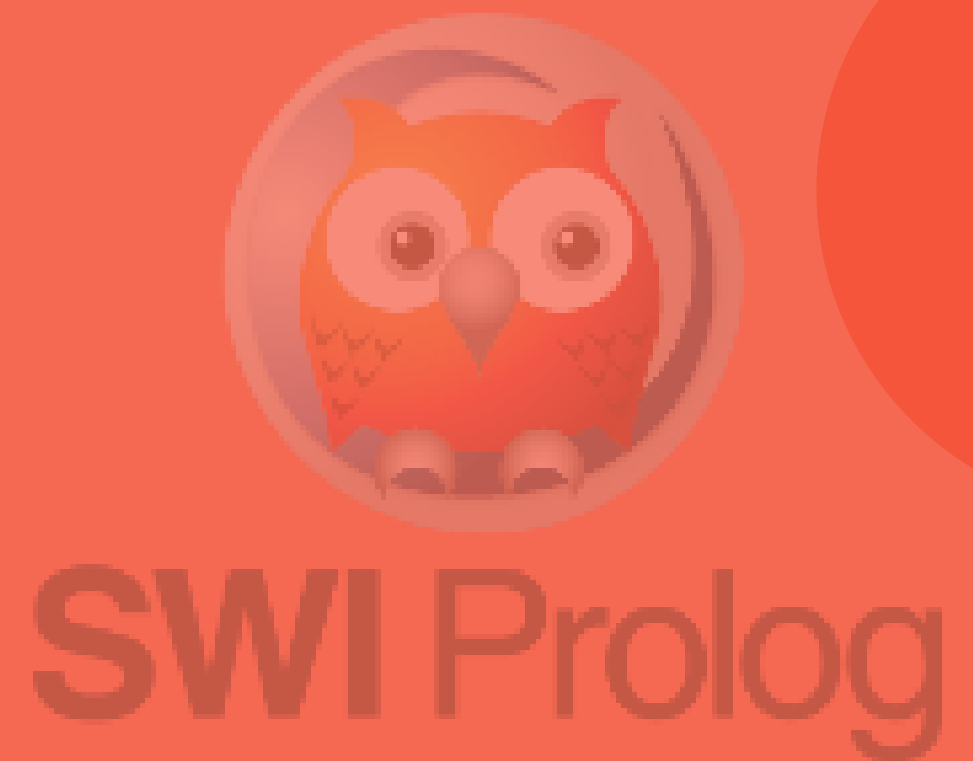
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- [fatorial].
true.

?- main.
Digite um numero:
|: 3
|: .

O Fatorial de 3 e igual a 6
true.

?-
```



Apêndice:

Gustavo

Achei bastante interessante a didática do Prolog, especialmente considerando o objetivo para o qual a linguagem foi criada. Ao compreender a história de sua origem e as motivações que levaram ao seu desenvolvimento, torna-se ainda mais surpreendente perceber o impacto que o Prolog teve, e ainda pode ter, no ensino da lógica e da programação. É impressionante que uma linguagem criada na década de 1970 continue sendo tão relevante em contextos educacionais e acadêmicos.

Apêndice:

Felipe

Acho muito interessante o funcionamento da linguagem, foi uma nova perspectiva. Definir uma base de conhecimento e fazer consultas em cima dela. O que mais me surpreendeu é as possibilidades das consultas, em que se pode passar dois valores para ver se é verdadeiro, ou passar um valor e uma variável, que mostra todas as possibilidades que atendem aquela consulta.

Além da linguagem não ter for, ou while, os testes condicionais serem tudo lógicos, não dá a sensação da linguagem ser verbosa. O que de primeira vista me deu uma certa dificuldade para entender a linguagem.



Apêndice:

Thiago

A parte que mais gostei em Prolog foram as definições de fatos e regras, pois mostra uma das partes mais usadas e interessantes do Prolog. Os fatos permitem registrar verdades simples, enquanto as regras mostram como essas verdades se relacionam. O interessante é que, a partir disso, o Prolog consegue deduzir novas informações sozinho, sem precisar que a gente explique passo a passo. Essa forma de raciocinar por meio de lógica foi o que achei mais atrativo na linguagem.



Sem mais **PROLOnGações**....

Muito Obrigado!



Referências Bibliográficas

- SWI-PROLOG. SWI-Prolog: ambiente de desenvolvimento Prolog. SWI-Prolog, 2025. Disponível em: <https://www.swi-prolog.org/>
- MERCURY. Mercury Language Reference Manual. Disponível em: <https://mercurylang.org/documentation/documentation.html>. Acesso em: 29 set. 2025.
- CIAO. Documentation — The Ciao Prolog system. Disponível em: <https://ciao-lang.org/documentation.html>. Acesso em: 29 set. 2025.
- LOGTALK. Documentation. Disponível em: <https://logtalk.org/documentation.html>. Acesso em: 29 set. 2025.
- VISUAL PROLOG. Fundamental Visual Prolog. Disponível em: https://wiki.visual-prolog.com/index.php?title=Fundamental_Visual_Prolog. Acesso em: 29 set. 2025.
- FEIGENBAUM, Edward A. The Japanese national Fifth Generation project. Artificial Intelligence, v. 59, n. 1, p. 1–22, 1993. DOI: 10.1016/0167-739X(93)90038-3
- RESEARCHAIN. A verdadeira ambição do projeto de computador de quinta geração: como o Japão está tentando liderar a era da IA? Disponível em: https://researchain.net/pt/fact/the-true-ambition-of-the-fifth-generation-computer-project-how-is-japan-trying-to-lead-the-ai-era#google_vignette. Acesso em: 1 out. 2025.
- História da Computação. (2023, 5 de março). Alain Colmerauer – vida e contribuições [Vídeo]. YouTube. https://www.youtube.com/watch?v=74lg_QKndvE
- MEI, Zhiwen. Programming in Prolog. Sacramento: California State University, [s.d.].
- SILVA, Marcelo. Apostila de Prolog. Uberlândia: Universidade Federal de Uberlândia – FACOM, [s.d.].